Elliptic Curve Cryptography (ECC)

- **What is ECC?**
  - ECC stands for Elliptic Curve Cryptography.
  - It is a **public-key cryptographic system**
  - Uses
    i. **Private key** (secret)
    ii. **Public key** (shared)
  - Security is based on hard mathematical problems over elliptic curves

ECC vs RSA: Key Size & Security

- ECC provides **equivalent security with much smaller keys**
- Smaller keys → faster computation, less storage, less bandwidth

Breaking 256-bit ECC requires roughly as much work as breaking 3072-bit RSA. So you get the same security in fewer bits.

ECC Key Size	RSA Key Size (Approx.)
256	3072
384	7680

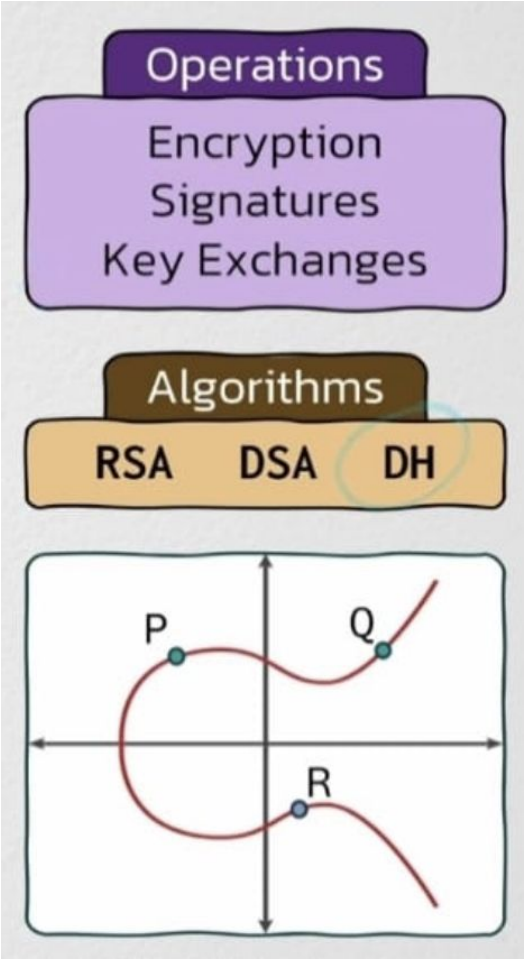
Elliptic Curve Cryptography (ECC):

- Traditional crypto uses *positive integers*
  - 2, 5, 37, 8954, 23462394, etc…

- ECC does the *same math* using points on an **Elliptic Curve**

- **RSA, DSA, DH have Elliptic Curve variants**:
  - **ECDSA → signature**
  - **ECDH → key exchange**
  - **ECIES → Encryption**

- **ECC is more secure and requires smaller keys**



Repeating this gives G → 2G → 3G → … → nG

This is **scalar multiplication**:

$$nG = \underbrace{G + G + \cdots + G}_{n \text{ times}}$$

- n = private key
- nG = public key

Given:

- G
- nG

Finding n is **computationally infeasible**
→ this is **ECDLP**

This is the **core security** of ECC.

How ECC Works

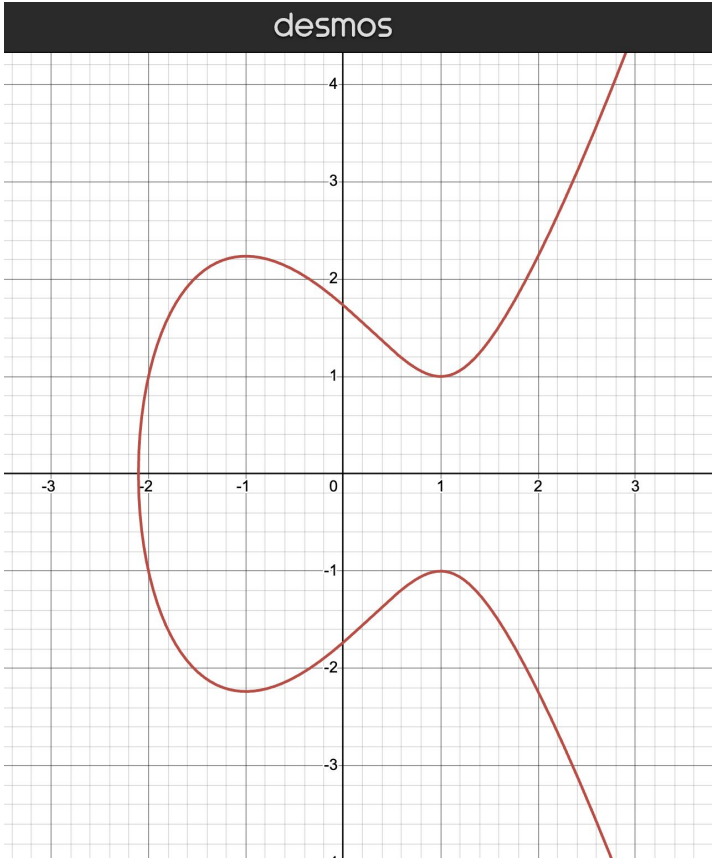
- **The Elliptic Curve**
  - **Symmetric about the x-axis**.
  - An elliptic curve is the set of all (x, y) satisfying:
    i. $y^2 = x^3 + ax + b$
- **The "Dot Function"**
  - Elliptic curve point addition.
    1. Draw a line through points P and G
    2. It intersects the curve at a third point -C
    3. Reflect vertically to get C=P+G
  - This process is **repeated "n" number of times**, resulting in an end point (Z).

STEP 0

Plot the curve: $y^2 = x^3 + ax + b$

We use $y^2 = x^3 - 3x + 3$ ($a = -3$, $b = 3$) throughout this example.

```
Discriminant check (must ≠ 0):
4a³ + 27b²
= 4(−3)³ + 27(3)²
= 4(−27) + 27(9)
= −108 + 243
= 135 ≠ 0 (smooth, non-singular - no self-intersections)
```

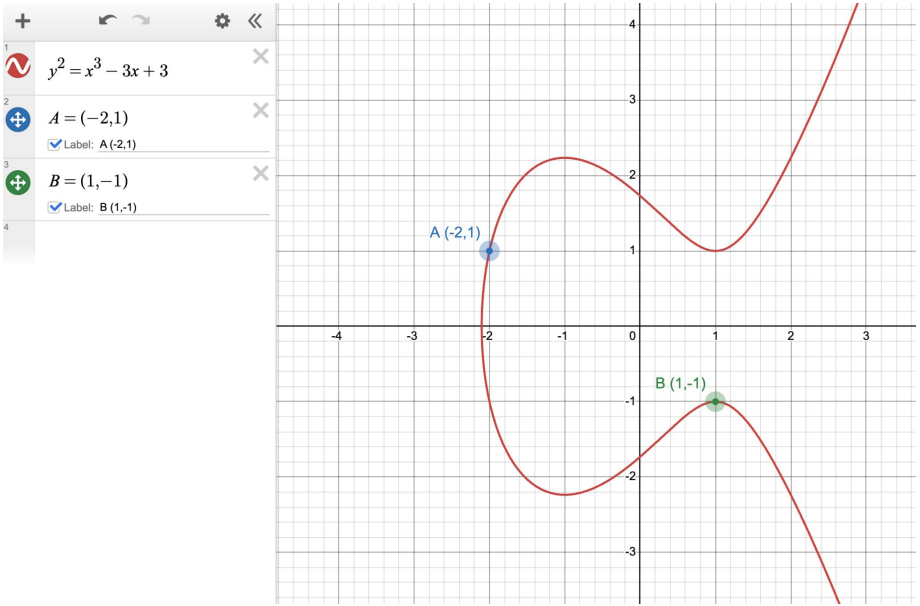


STEP 1: Place points A and B on the curve

A = (−2, 1)

B = (1, −1)

```
Verify A = (-2, 1):
y² = x³ - 3x + 3
(1)² = (-2)³ - 3(-2) + 3
  1 = -8 + 6 + 3 = 1  ✓

Verify B = (1, -1):
(-1)² = (1)³ - 3(1) + 3
   1  = 1 - 3 + 3 = 1  ✓
```



STEP 2: A dot B → C (draw secant, find 3rd intersection, reflect)

1. Slope of secant line through A, B:

$$s = (y_A - y_B) / (x_A - x_B)$$
$$= (1 - (-1)) / (-2 - 1)$$
$$= 2 / (-3)$$
$$= -2/3$$

2. x-coordinate of C:

$$C_x = s^2 - A_x - B_x$$
$$= (-2/3)^2 - (-2) - 1$$
$$= 4/9 + 2 - 1$$
$$= 4/9 + 1$$
$$= 13/9 \approx 1.444$$

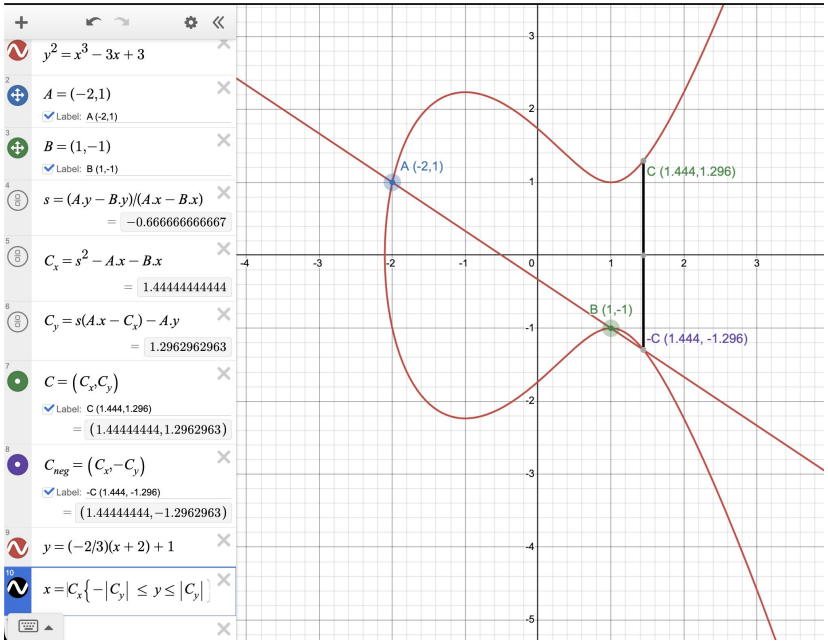
3. y-coordinate of C:

$$C_y = s(A_x - C_x) - A_y$$
$$= (-2/3)(-2 - 13/9) - 1$$
$$= (-2/3)(-31/9) - 1$$
$$= 62/27 - 1$$
$$= 35/27 \approx 1.296$$

C = A + B = (13/9, 35/27) ≈ (1.444, 1.296)

Verify C on curve:

$$y^2 = x^3 - 3x + 3$$
$$(35/27)^2 = (13/9)^3 - 3(13/9) + 3$$

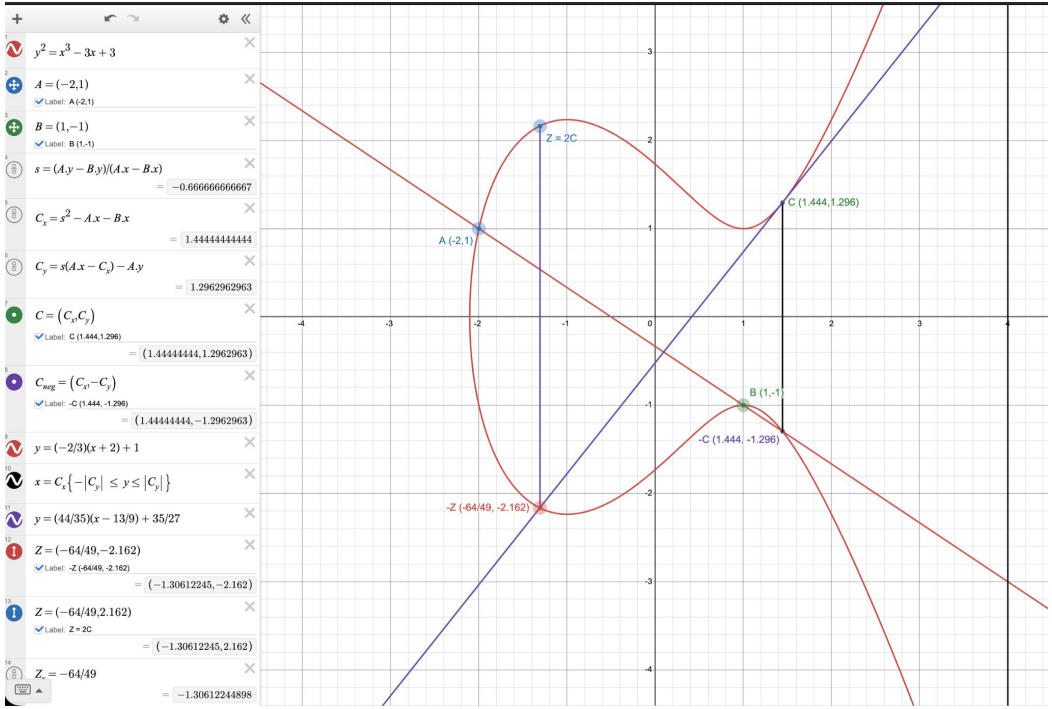


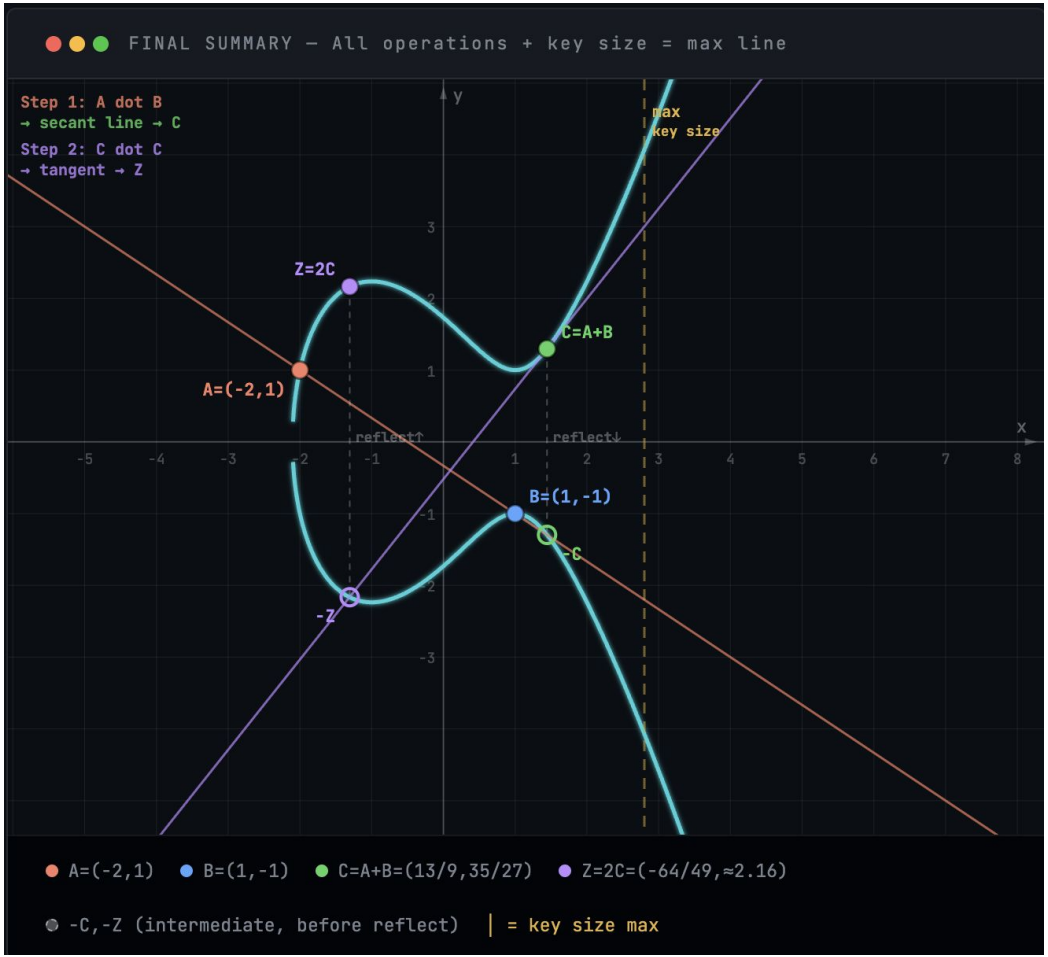
STEP 3: (C dot C = 2C, tangent at C) → Z

```
Tangent slope at C (from implicit differentiation):
s = (3(C_x)² + a) / (2C_y)
  = (3 × (13/9)² + (-3)) / (2 × 35/27)
  = (3 × 169/81 - 3) / (70/27)
  = 44/35  ≈ 1.257

x-coordinate of Z:
x_Z = s² - 2x_C
    = (44/35)² - 2(13/9)
    = 1936/1225 - 26/9
    = -1600/1225 = -64/49  ≈ -1.306

y-coordinate of Z:
y_Z = s(x_C - x_Z) - y_C
    = (44/35)(13/9 - (-64/49)) - 35/27
    ≈ 1.257 × (1.444 + 1.306) - 1.296
    ≈ 1.257 × 2.751 - 1.296
    ≈ 3.458 - 1.296
    ≈ 2.162
```

Z = 2C ≈ (-1.306, 2.162) [upper-left area]

Verified on curve ✓

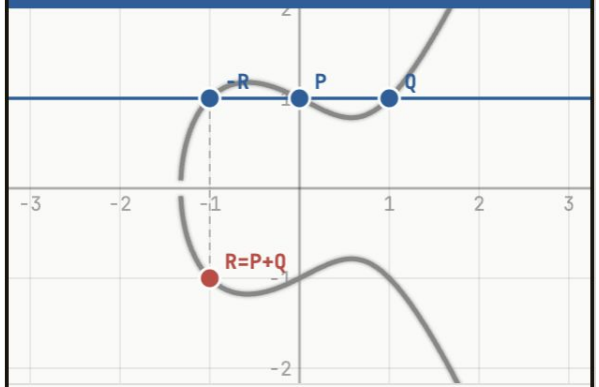



FINAL SUMMARY — All operations + key size = max line
Step 1: A dot B
→ secant line → C
Step 2: C dot C
→ tangent → Z
max
key size
Z=2C
C=A+B
A=(-2,1)
B=(1,-1)
-C
-Z
reflect↑
reflect↓
A=(-2,1)
B=(1,-1)
C=A+B=(13/9,35/27)
Z=2C=(-64/49,≈2.16)
-C,-Z (intermediate, before reflect)
= key size max

All 3 cases of point addition

There are exactly three distinct cases we encounter on an elliptic curve:

CASE A — P ≠ Q (SECANT)

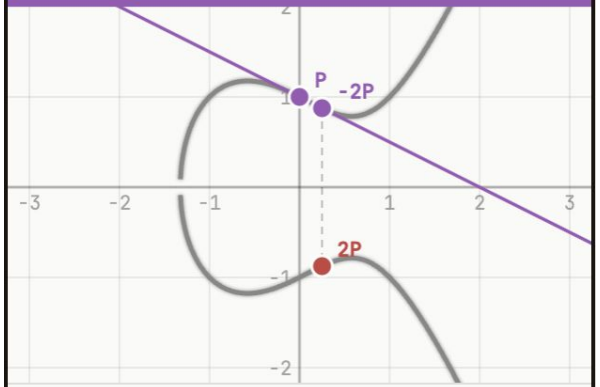


Draw secant line through P and Q.

Hit curve at third point −R. Reflect to get R = P+Q.

$s = (y_P - y_Q)/(x_P - x_Q)$

CASE B — P = Q (TANGENT)

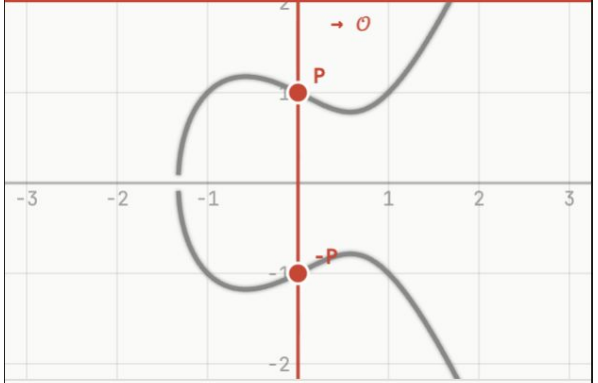


Draw tangent line at P.

Hit curve at one other point −R. Reflect to get 2P.

$s = (3x_P^2 + a)/(2y_P)$

CASE C — P + (-P) = $\mathcal{O}$



Vertical line through P and −P.

Never hits a finite third point → Point at Infinity $\mathcal{O}$.

P + (-P) = $\mathcal{O}$ (identity)

How the SRS is built using ECC

Generate secret τ(tau)

Compute powers of τ(tau) on the curve

instead of raw numbers, compute elliptic curve points using τ:

$G,\ \tau G,\ \tau^2 G,\ \tau^3 G,\ \ldots$

Destroy τ forever

Publish the SRS

The list $[G, \tau G, \tau^2 G, \tau^3 G, \ldots]$ is public. This is the Structured Reference String.

Use SRS to commit to a polynomial

For $P(x) = 3x^2 + 2x + 1$, the commitment is: $3 \cdot (\tau^2 G) + 2 \cdot (\tau G) + 1 \cdot G$

The full ZK pipeline

Neural network weights
The private data (what you're proving)

Constraints
Each operation becomes a constraint: "output = a × b"

Polynomials
Constraints are encoded as polynomials

Evaluate at τ using SRS
Use $[\tau^0 G, \tau^1 G, \tau^2 G \ldots]$ to evaluate without knowing τ

Elliptic curve points = Proof
A compact proof that can be verified in milliseconds

Verifying Key
Published so anyone can verify without seeing the weights